Chitkara

PAPER OF CSE-2025 - PYTHON TEST 5 EXAM

Q1. Battle of the Titans

In a mythical land, colossal Titans rise to challenge the kingdoms. Each Titan has an energy level representing its strength. Warriors strike with powerful hammers until a Titan's energy drops to zero. Your task is to simulate the battle and determine how many Titans fall and the total hammer strikes needed.

Instruction/Task:

Calculate:

1. Total Titans defeated.
2. Total hammer strikes used.

Note:

If the number of strikes results in a decimal value, convert it to the **next whole number**.

For example:

- If you get **2.1**, it becomes **3**
- If you get **2.9**, it also becomes **3**

This means you must **round up** every decimal value.

Use the `ceil()` function to perform this rounding.

Input Format:

- Enter Titan energy levels as a list.
- Enter hammer power per strike as an integer.

Output Format:

- Titans defeated: X
- Total hammer strikes used: Y

Sample Input:

```
[120, 95, 150, 80, 60]
30
```

Sample Output:

```
5
```

```
18
```

Calculate strikes for each Titan:

- **Titan 1:** 150 → (150 / 40) = 5 strikes
- **Titan 2:** 95 → (95 / 40) = 3 strikes
- **Titan 3:** 60 → (60 / 40) = 2 strikes
- **Titan 4:** 200 → (200 / 40) = 5 strikes
- **Titan 5:** 85 → (85 / 40) = 3 strikes
- **Titan 6:** 120 → (120 / 40) = 3 strikes
- **Titan 7:** 40 → (40 / 40) = 1 strike

Total Titans defeated = 7, total strikes = 20

ANSWER:

Test case 1 :

Input:
[120, 95, 150, 80, 60]
30
Output:
5
18

Test case 2 :

Input:
[5, 5, 5]
10
Output:
3
3

Test case 3 :

Input:
[0, 10, 20]
10
Output:
2
3

Test case 4 :

Input:
[]
10
Output:
0
0

Test case 5 :

Input:

[25, 45, 70]
5
Output:
3
28

Test case 6 :

Input:
[100]
25
Output:
1
4

Q2. Monster Hunt in Dark Forest

You are a hunter in the dark forest where dangerous monsters roam. Each monster has a health level. You attack monsters with your sword until their health reaches zero.

Instruction/Task:

Calculate:

1. Total monsters defeated.
2. Total sword swings used.

Note:

If the number of strikes results in a decimal value, convert it to the **next whole number**.

For example:

- If you get **2.1**, it becomes **3**
- If you get **2.9**, it also becomes **3**

This means you must **round up** every decimal value.

Use the `ceil()` function to perform this rounding.

Input Format:

- List of integers → monster healths
- Integer → sword power per swing

Output Format:

- Monsters defeated: X
- Total sword swings used: Y

Sample Input:

```
[25, 90, 40, 10]
15
```

Sample Output:

```
4
12
```

Explanation:

- Monster 1: 25 → (25 / 15) = 2 swings
- Monster 2: 90 → (90 / 15) = 6 swings
- Monster 3: 40 → (40 / 15) = 3 swings
- Monster 4: 10 → (10 / 15) = 1 swing

Total monsters defeated: 4

Total sword swings used: 12

ANSWER:

Test case 1 :

Input:
[25, 90, 40, 10]
15
Output:
4
12

Test case 2 :

Input:
[5, 5, 5]
10
Output:
3
3

Test case 3 :

Input:
[0, 10, 20]
10
Output:
2
3

Test case 4 :

Input:
[]
10
Output:
0

0

Test case 5 :

Input:
[25, 45, 70]
5
Output:
3
28

Test case 6 :

Input:
[100]
25
Output:
1
4

Q3. Alien Planet Defense

On an alien planet, you must defend your base from alien ships. Each ship has shield points. Your plasma cannon reduces shield points. When shield drops to zero or below, the ship is destroyed.

Instruction/Task:

Determine:

1. Total aliens destroyed.
2. Total plasma shots fired.

Input Format:

- List of integers → alien shield points
- Integer → plasma cannon power per shot

Output Format:

- Aliens destroyed: X
- Total shots fired: Y

```
[50, 60, 80]
25
```

Sample Output:

```
3
9
```

Explanation:

- Alien 1 → 2 shots

- Alien 2 → 3 shots
- Alien 3 → 4 shots

Total aliens destroyed = 3, shots fired = 9

ANSWER:

Test case 1 :

Input:
[50, 60, 80]
25
Output:
3
9

Test case 2 :

Input:
[5, 5, 5]
10
Output:
3
3

Test case 3 :

Input:
[0, 10, 20]
10
Output:
2
3

Test case 4 :

Input:
[]
10
Output:
0
0

Test case 5 :

Input:
[25, 45, 70]
5
Output:
3
28

Test case 6 :

Input:

[100]
25
Output:
1
4

Q4. Event Ticket ID Checker

Your college is organising an event with entry tickets. They will be using new automatic ticketing system.

This ticketing system uses IDs in the format:

TKT-DDLL (where DD is 2 digits, LL is 2 uppercase letters).

The system needs to validate all entered IDs.

Rules:

- The code must start with "**TKT-**".
- If not, raise an exception → "**Bad Input**".
- The next segment must be exactly 2 digits followed by 2 uppercase letters.
- If this segment is wrong (contains only digits or alphabets, wrong length, wrong characters sequence) → raise an exception → "**Bad Format**".
- If both are correct → return "**Valid Ticket**".

Your Task:

Write a function using exception handling to validate the ticket ID and return the final status message.

Input Format:

A single string representing the ticket ID.

Output Format:

Print one of the following messages: "Ticket Verified", "Bad Input", "Bad Format"

Sample Test case 1:

Input:

```
"TKT-12AB"
```

Output:

```
Valid Ticket
```

Sample Test case 2:

Input:

```
"TKT-A12B"
```

Output:

```
Bad Format
```

ANSWER:

Test case 1 :

Input:
TKT-12AB
Output:
Valid Ticket

Test case 2 :

Input:
TKT-A12B
Output:
Bad Format

Test case 3 :

Input:
TKT-AA12
Output:
Bad Format

Test case 4 :

Input:
TKT-1121
Output:
Bad Format

Test case 5 :

Input:
TK-1456
Output:
Bad Input

Test case 6 :

Input:
TKT-T12E3
Output:
Bad Format

Q5. Chemical Batch Identifier

A laboratory uses batch identifiers following the format:

CHEM-LDDDD (where L is one uppercase letter, DDDD is 4 digits).

Faulty inputs must be filtered.

Rules:

- The code must start with "**CHEM-**".
- If not, raise an exception → "**Protocol Error**"
- The next part must be exactly one uppercase letter, followed by exactly four digits.
- If this segment is wrong (contains only digits or alphabets, wrong length, wrong characters sequence) → raise an exception → "**Format Error**"
- If both are correct → return "**Batch Ready**".

Your Task:

Write a function using exception handling to validate the chemical batch identifier and return the status message.

Input Format:

A single string representing the batch identifier.

Output Format:

Print one of the following messages: "Batch Ready", "Protocol Error", "Format Error"

Sample Test case 1:

Input:

```
"CHEM-Z0001"
```

Output:

```
Batch Ready
```

Sample Test case 2:

Input:

```
"CHEM-1234A"
```

Output:

```
Format Error
```

ANSWER:

Test case 1 :

Input:
CHEM-Z0001
Output:
Batch Ready

Test case 2 :

Input:
CHEM-1234A
Output:
Format Error

Test case 3 :

Input:
CHEM-000011
Output:
Format Error

Test case 4 :

Input:
CHEZ-A0001
Output:
Protocol Error

Test case 5 :

Input:
CHEM-ZZ000
Output:
Format Error

Test case 6 :

Input:
CHEM-T12E3
Output:
Format Error

Q6. Satellite Telemetry Code Check

A satellite control system verifies telemetry codes.

The format is:

TLM-NLLL (where N is 1 digit, LLL is 3 uppercase letters).

Invalid codes must be identified.

Rules:

- The code must start with "**TLM-**".
- If it doesn't, raise an exception → "**Code Missing**"
- The next part must be exactly one digits, followed by exactly three uppercase letter.
- If this segment is wrong (contains only digits or alphabets, wrong length, wrong characters sequence) → raise an exception → "**Code Corrupt**"
- If all are correct → return "**Code Accepted**".

Your Task:

Write a function using exceptions to check the telemetry code and return the final status message.

Input Format:

A single string representing the telemetry code.

Output Format:

Show one of the following messages according to input: "Code Accepted", "Code Missing", "Code Corrupt".

Sample Test case 1:

Input:

```
"TLM-9ABC"
```

Output:

```
Code Accepted
```

Sample Test case 2:

Input:

```
"TLM-A9BC"
```

Output:

```
Code Corrupt
```

ANSWER:

Test case 1 :

Input:
TLM-9ABC
Output:
Code Accepted

Test case 2 :

Input:
TLM-A9BC
Output:
Code Corrupt

Test case 3 :

Input:
TLM-AABC
Output:
Code Corrupt

Test case 4 :

Input:
S1-N122
Output:
Code Missing

Test case 5 :

Input:
TLM-9ABC999
Output:
Code Corrupt

Test case 6 :

Input:
TLM-A9BC
Output:
Code Corrupt

Q7. Airline Crew Pay Regulation System

An airline tracks crew payments:

```
{'CrewMember':[BasePay, Bonus%]}
```

Due to aviation regulations, crew pay must be adjusted.

Rules:

- Final pay = `Base + (Base × Bonus%) / 100`
- Remove crew whose final pay is **less than 20000**
- If final pay is **greater than 50000**, add them to `highEarners`
- Return `(updated_data, highEarners)`

Input Format:

```
{Name: [BasePay, Bonus%]}
```

Output Format:

```
(updated_data, highEarners)
```

Sample Test Case 1:

Input:

```
{'Pilot':[40000,25],'Crew1':[18000,10],'Crew2':[25000,15]}
```

Output:

```
({'Pilot':50000.0, 'Crew2':28750.0}, {})
```

Sample Test Case 2:

Input:

```
{'Pilot1':[30000,30],'CrewA':[16000,25],'CrewB':[55000,10]}
```

Output:

```
({'Pilot1':39000.0, 'CrewA':20000.0, 'CrewB':60500.0}, {'CrewB':60500.0})
```

ANSWER:

Test case 1 :

Input:
{'Pilot':[40000,25],'Crew1':[18000,10],'Crew2':[25000,15]}
Output:
({'Pilot': 50000.0, 'Crew2': 28750.0}, {})

Test case 2 :

Input:
{'A':[30000,10],'B':[10000,20]}
Output:
({'A': 33000.0}, {})

Test case 3 :

Input:
{'X':[45000,15],'Y':[20000,0]}
Output:
({'X': 51750.0, 'Y': 20000.0}, {'X': 51750.0})

Test case 4 :

Input:
{'John':[18000,5],'Lia':[22000,0]}
Output:
({'Lia': 22000.0}, {})

Test case 5 :

Input:
{'P':[50000,5],'Q':[10000,50]}

Output:
({'P': 52500.0}, {'P': 52500.0})

Test case 6 :

Input:
{'Anu':[40000,40],'Tia':[26000,10]}

Output:
({'Anu': 56000.0, 'Tia': 28600.0}, {'Anu': 56000.0})

Q8. Tech Startup Salary Normalizer

A startup stores employee compensation as:

```
{'Employee':[BaseSalary, Bonus%]}
```

Due to funding issues, they need to normalize salaries.

Rules:

- Final salary = `Base + (Base × Bonus%) / 100`
- Remove employees whose final salary is **less than 20000**
- If final salary is **greater than 50000**, add them to `highEarners`
- Return `(updated_data, highEarners)`

Input Format:

```
{Name: [BaseSalary, Bonus%]}
```

Output Format:

```
(updated_data, highEarners)
```

Sample Test Case 1:

Input:

```
{'Emp1':[18000,50],'Emp2':[22000,10],'Emp3':[45000,30]}
```

Output:

```
({'Emp1':27000.0, 'Emp2':24200.0, 'Emp3':58500.0}, {'Emp3':58500.0})
```

Sample Test Case 2:

Input:

```
{'Dev1':[15000,20],'Dev2':[21000,0]}
```

Output:

```
({'Dev2':21000.0}, {})
```

ANSWER:

Test case 1 :

Input:
{'Emp1':[18000,50],'Emp2':[22000,10],'Emp3':[45000,30]}
Output:
({'Emp1': 27000.0, 'Emp2': 24200.0, 'Emp3': 58500.0}, {'Emp3': 58500.0})

Test case 2 :

Input:
{'A':[30000,10],'B':[10000,20]}
Output:
({'A': 33000.0}, {})

Test case 3 :

Input:
{'X':[45000,15],'Y':[20000,0]}
Output:
({'X': 51750.0, 'Y': 20000.0}, {'X': 51750.0})

Test case 4 :

Input:
{'John':[18000,5],'Lia':[22000,0]}
Output:
({'Lia': 22000.0}, {})

Test case 5 :

Input:
{'P':[50000,5],'Q':[10000,50]}

Output:
({'P': 52500.0}, {'P': 52500.0})

Test case 6 :

Input:
{'Anu':[40000,40],'Tia':[26000,10]}
Output:
({'Anu': 56000.0, 'Tia': 28600.0}, {'Anu': 56000.0})

Q9. Sports Tournament Prize Evaluator

A sports board maintains player rewards:

```
{'Player':[BaseReward, Bonus%]}
```

Due to sponsorship constraints, rewards must be recalculated.

Rules:

- Final reward = `Base + (Base × Bonus%) / 100`
- Remove players whose final reward is **less than 20000**
- If final reward is **greater than 50000**, add them to `highEarners`
- Return `(updated_data, highEarners)`

Input Format:

```
{Name: [BaseReward, Bonus%]}
```

Output Format:

```
(updated_data, highEarners)
```

Sample Test Case 1:

Input:

```
{'Player1':[25000,10],'Player2':[15000,5],'Player3':[60000,20]}
```

Output:

```
({'Player1':27500.0, 'Player3':72000.0}, {'Player3':72000.0})
```

Sample Test Case 2:

Input:

```
{'PlayerA':[19500,5],'PlayerB':[22000,0]}
```

Output:

```
({'PlayerA':20475.0, 'PlayerB':22000.0}, {})
```

ANSWER:

Test case 1 :

Input:
{'Player1':[25000,10],'Player2':[15000,5],'Player3':[60000,20]}
Output:
({'Player1': 27500.0, 'Player3': 72000.0}, {'Player3': 72000.0})

Test case 2 :

Input:
{'A':[30000,10],'B':[10000,20]}
Output:
({'A': 33000.0}, {})

Test case 3 :

Input:

{'X':[45000,15],'Y':[20000,0]}
Output:
({'X': 51750.0, 'Y': 20000.0}, {'X': 51750.0})

Test case 4 :

Input:
{'John':[18000,5],'Lia':[22000,0]}
Output:
({'Lia': 22000.0}, {})

Test case 5 :

Input:
{'P':[50000,5],'Q':[10000,50]}

Output:
({'P': 52500.0}, {'P': 52500.0})

Test case 6 :

Input:
{'Anu':[40000,40],'Tia':[26000,10]}
Output:
({'Anu': 56000.0, 'Tia': 28600.0}, {'Anu': 56000.0})